

Formulations and a Lagrangian relaxation approach for the prize collecting traveling salesman problem

Guido Pantuza Jr.^{a,b} and Mauricio C. de Souza^{c,*} 

^a*Programa de Pós-Graduação em Engenharia de Produção, Universidade Federal de Minas Gerais, Av. Antônio Carlos, 6627, Belo Horizonte 31270-901, Brazil*

^b*Instituto Federal de Minas Gerais, Avenida Minas Gerais, 5189, Governador Valadares, MG 35057-760, Brazil*

^c*Departamento de Engenharia de Produção, Universidade Federal de Minas Gerais, Av. Antônio Carlos, 6627, Belo Horizonte 31270-901, Brazil*

E-mail: guido.junior@ifmg.edu.br [Pantuza]; prof.mauriciodesouza@gmail.com [de Souza]

Received 9 June 2021; received in revised form 18 July 2021; accepted 19 July 2021

Abstract

The prize collecting traveling salesman problem (PCTSP) is a generalization of the traveling salesman problem (TSP) in which a tour starting at a root node must visit a subset of nodes to collect a prescribed amount of total prize minimizing the summation of travel costs and penalty costs associated with nonvisited nodes. We propose new formulations and compare them with adaptations to the PCTSP of strong formulations from the literature for the asymmetric TSP. One of the formulations proposed in this paper provided the tighter linear relaxation bounds on computational experiments based on benchmark instances for the asymmetric TSP. We then develop a Lagrangian relaxation approach embedding heuristic schemes to obtain both lower and upper bounds. The proposed heuristic has shown to be successful in obtaining optimal or near-optimal solutions with information from the Lagrangian subproblems. Our Lagrangian approach can thereupon provide small optimality gaps, being an efficient alternative to bound optimal values of large instances.

Keywords: Lagrangian relaxation; prize collecting traveling salesman problem; network optimization

1. Introduction

Given a directed graph $G = (V, A)$, the prize collecting traveling salesman problem (PCTSP) is a generalization of the traveling salesman problem (TSP) in which a tour starting at a root node $r \in V$ must visit a subset of nodes to collect a prescribed amount of total prize. A nonnegative prize p_i is collected if a node $i \in V \setminus \{r\}$ is visited in the tour, and a penalty w_i is incurred otherwise. Let c_{ij} be the cost of traveling from node i to j , the objective is to minimize the summation of travel costs and penalties incurred in a tour visiting each node at most once and collecting at least

*Corresponding author.

© 2021 The Authors.

International Transactions in Operational Research © 2021 International Federation of Operational Research Societies

Published by John Wiley & Sons Ltd, 9600 Garsington Road, Oxford OX4 2DQ, UK and 350 Main St, Malden, MA02148, USA.

a total prize of g . In this paper, we consider the asymmetric version of the PCTSP with general costs, that is, the costs are not necessarily assumed to satisfy the triangle inequality. According to Cirasella et al. (2001), heuristics for the asymmetric TSP are not as powerful to solve the asymmetric version of the problem as the heuristics tailored for the symmetric version are in their context. We also remark in the literature an effort that has been undertaken to develop strong formulations for the more challenging asymmetric version of the TSP (see Sherali et al., 2006; Oncan et al., 2009; Godinho et al., 2011; Roberti and Toth, 2012). In view of this, we believe that it is worth studying formulations and bounding procedures for the asymmetric version of the PCTSP with general costs.

The PCTSP has been used to address scheduling problems arising in the metallurgical industry. Balas and Martin (1985) were the first to formulate the PCTSP as a model for scheduling the daily operation of a steel rolling mill. The problem addressed by the authors consisted of choosing and scheduling a subset of slabs from an inventory to satisfy a lower bound on total weight minimizing the makespan. Lopez et al. (1998) addressed the hot strip mill production scheduling problem. The authors dealt with a hard problem with multiple and conflicting objectives and constraints modeling it as a generalization of the PCTSP. Zhang and Tang (2005) and Tang and Wang (2008) considered a capacitated version of PCTSP when dealing with the color-coating production scheduling in a cold rolling mill.

Balas (1989) discussed polyhedral properties of the PCTSP, identifying facet-defining inequalities derived from facets of the traveling salesman polytope and the knapsack polytope. The results were then extended by Balas (1995) for deriving a facet from any facet-defining inequality for the asymmetric traveling salesman polytope. Fischetti and Toth (1988) developed an additive procedure combining several lower bounds and use it in a branch-and-bound algorithm for the PCTSP. The additive procedure produces in general a better lower bound than taking individually each of them. Dell'Amico et al. (1995) considered the PCTSP and the profitable tour problem as well, where it is not imposed to collect a minimum prize and there is no predefined root. Their results were used by Dell'Amico et al. (1998) in a Lagrangian relaxation scheme yielding an asymmetric profitable tour problem as a subproblem. Then, a Lagrangian heuristic based on an extension and collapse procedure was proposed. Bérubé et al. (2009) proposed a branch-and-cut algorithm for the symmetric version of the PCTSP. The authors exploited valid inequalities derived from the traveling salesman polytope and the knapsack polytope, and also developed cuts based on polyhedral results for the orienteering problem. The symmetric version of the PCTSP has also been addressed recently by Clímaco et al. (2021) with MIP-based heuristics and a branch-and-cut algorithm. The PCTSP has been addressed with metaheuristics by Chaves and Lorena (2008) who proposed a hybrid metaheuristic, known as clustering search, where detected promising areas of the search space are explored through local search methods, and by Pedro et al. (2013) who proposed a tabu search approach. Bienstock et al. (1993) were the first to address the PCTSP with an approximation algorithm. Since then, other approximation algorithms have been proposed for the PCTSP in the literature, see for instance the more recent works of Chan et al. (2020) and Gimadi and Tsidulko (2020).

We present six formulations for the PCTSP: two of them use classical ideas of subtour elimination and multicommodity formulations; two of them are adaptations to the PCTSP of strong formulations for the asymmetric TSP due to Sherali et al. (2006) and Godinho et al. (2011); and two of them are proposed in this paper. We propose, on one hand, a strong formulation denoted by enhanced path-node giving tight linear relaxation bounds, and on the other hand, a formulation

denoted by prize-single commodity whose linear relaxation is solved very fast using little memory (giving weak linear relaxation bounds though). The former provided the best linear relaxation bounds among the six formulations analyzed on instances based on benchmark asymmetric TSP instances, whereas only with the latter we were able to get optimal solutions for all instances running an optimization package without time limit. The enhanced path-node formulation is used to develop a Lagrangian relaxation approach embedding heuristic schemes to obtain both lower and upper bounds. Our Lagrangian approach uses valid inequalities from the literature in a relax-and-cut fashion (Guignard, 2003; Lucena, 2005) to improve lower bounds. Our computational experiments have shown that the Lagrangian approach provides small optimality gaps, being an efficient alternative to bound optimal values of large instances.

The paper is structured as follows. In Section 2, we present the six formulations analyzed in this study. In Section 3, we describe the Lagrangian relaxation approach. In Section 4, we report computational experiments, and in Section 5 we present concluding remarks.

2. Formulations

The underlying graph G is assumed to be directed and complete. A binary variable y_i assumes 1 if node i is visited (y_r is set to 1), and 0 otherwise; and x_{ij} assumes 1 if arc (i, j) is used in the tour, and 0 otherwise. The objective function is then given by

$$\min \sum_{(i,j) \in A} c_{ij}x_{ij} + \sum_{i \in V} w_i(1 - y_i). \quad (1)$$

The following constraints define the set of disjoint subtours in which the total prize g is collected by the visited nodes; see Balas (1989) and Fischetti and Toth (1988):

$$\sum_{j \in V \setminus \{i\}} x_{ij} = y_i \quad \forall i \in V \quad (2)$$

$$\sum_{i \in V \setminus \{j\}} x_{ij} = y_j \quad \forall j \in V \quad (3)$$

$$\sum_{i \in V \setminus \{r\}} p_i y_i \geq g \quad (4)$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \quad (5)$$

$$y_i \in \{0, 1\} \quad \forall i \in V \setminus \{r\}. \quad (6)$$

In this section, we present different sets of constraints that added to (2)–(6) define a valid formulation for the PCTSP, that is, ensure that every node selected to be visited can be reached from node r .

2.1. Subtour elimination

Balas (1989) formulated the PCTSP using subtour elimination constraints. The formulation, denoted by GSEC, adds the following constraints to (2)–(6):

$$\sum_{i \in S} \sum_{j \in S \setminus \{i\}} x_{ij} \leq \sum_{i \in S \setminus \{k\}} y_i \quad \forall S \subset V, 2 \leq |S| \leq |V| - 1, r \in V \setminus S, \forall k \in S. \quad (7)$$

Given $(\bar{x}, \bar{y}) \in [0, 1]^{|A| \times (|V|-1)}$ satisfying constraints (2)–(4), violated constraints (7), if any, can be obtained by solving $|\bar{V}|$ maximum flow problems, where $\bar{V} = \{v \in V \setminus \{r\} : \bar{y}_v > 0\}$; see, for instance, Fischetti et al. (1998) and Bérubé et al. (2009). For each $v \in \bar{V}$, the maximum flow between r and v on a graph with set $\bar{V}$ of nodes and set $\bar{A} = \{(i, j) \in A : \bar{x}_{ij} > 0\}$ of arcs where $\bar{x}_{ij}$ is the capacity of arc $(i, j) \in \bar{A}$ is computed. The set S in the minimum-capacity cut containing node v gives a potentially violated subtour constraint (7). In our computational experiments, we used the implementation due to Dinitz (2006) for the maximum flow problem, available at <https://gist.github.com/gabber12/5257878>, whose complexity is $O(n^2m)$ for a graph with n nodes and m arcs.

It is possible to write valid formulations for the PCTSP using a polynomial number of subtour elimination constraints. However, we do not consider such formulations in this work since it has been shown they provide weak linear relaxation bounds for the asymmetric TSP; see Godinho et al. (2011) and Oncan et al. (2009).

2.2. Multicommodity flows

Multicommodity flow (MCF) formulations developed for the TSP can be easily adapted to the PCTSP. Claus (1984) proposed a formulation for the TSP using $|V| - 1$ commodities. In the PCTSP, only the nodes selected to make part of the tour send one unit of flow to the root. The flow variable f_{ij}^k assumes 1 if and only if node k is part of the tour and the commodity going from k to r flows on arc (i, j) . Then, the MCF formulation can be adapted for the PCTSP by adding the following constraints to (2)–(6):

$$\sum_{j \in V \setminus \{i\}} f_{ij}^k - \sum_{j \in V \setminus \{i\}} f_{ji}^k = \begin{cases} y_k & \text{if } k = i \\ 0 & \text{otherwise} \end{cases} \quad \forall k, i \in V \setminus \{r\} \quad (8)$$

$$f_{ij}^k \leq x_{ij} \quad \forall (i, j) \in A, \forall k \in V \setminus \{r\} \quad (9)$$

$$f_{ij}^k \geq 0 \quad \forall (i, j) \in A, \forall k \in V \setminus \{r\}. \quad (10)$$

2.3. Ordered path-based

Sherali et al. (2006) derived with the reformulation-linearization technique (RLT) (Sherali and Adams, 1990, 1994) formulations using precedence variables for the asymmetric TSP. Considering

the asymmetric TSP, let v_{ij} , $i, j \in V \setminus \{r\}$, $i \neq j$, a variable that assumes 1 if node i precedes node j in the tour. Applying the first-order RLT, the authors derived formulations with variable $t_{ij}^k = x_{ik}v_{kj}$, $i, j, k \in V \setminus \{r\}$, $i \neq j \neq k$, which assuming 1 means that arc (i, k) is part of the tour and node k precedes node j .

In this work, we adapt to the PCTSP, a formulation that provided the tightest linear relaxation bounds among the ones proposed in that paper, namely, ATSP6. Because in the context of the PCSTP, a node is not necessarily part of the tour, such an adaptation is not straightforward. Indeed, variables v_{ij} and t_{ij}^k can only assume 1 if nodes i , j , and k are all selected to be part of the tour. This fact leads to significant modifications when adapting the ATSP6 formulation to the PCTSP. The formulation, denoted by OPB', adds the following constraints to (2)–(6):

$$v_{ij} + v_{ji} \leq y_i \quad \forall i, j \in V \setminus \{r\}, i \neq j \quad (11)$$

$$v_{ij} + 1 - y_j \geq x_{ri} \quad \forall i, j \in V \setminus \{r\}, i \neq j \quad (12)$$

$$v_{ji} + 1 - y_j \geq x_{ir} \quad \forall i, j \in V \setminus \{r\}, i \neq j \quad (13)$$

$$v_{ij} + x_{ji} + v_{jk} + v_{ki} \leq 2 \quad \forall i, j, k \in V \setminus \{r\}, i \neq j \neq k \quad (14)$$

$$t_{ij}^k \leq x_{ik} \quad \forall i, j, k \in V \setminus \{r\}, k \neq j \neq i \quad (15)$$

$$\sum_{k \in V \setminus \{r, i, j\}} t_{ij}^k + x_{ij} = v_{ij} \quad \forall i, j \in V \setminus \{r\}, i \neq j \quad (16)$$

$$\sum_{i \in V \setminus \{r, k, j\}} t_{ij}^k + x_{rk} + y_j - 1 \leq v_{kj} \quad \forall k, j \in V \setminus \{r\}, k \neq j \quad (17)$$

$$\sum_{i \in V \setminus \{r, k, j\}} t_{ij}^k + x_{rk} - y_j + 1 \geq v_{kj} \quad \forall k, j \in V \setminus \{r\}, k \neq j \quad (18)$$

$$t_{ij}^k \geq 0 \quad \forall i, j, k \in V \setminus \{r\}, k \neq j \neq i. \quad (19)$$

Constraints (11) assert that for given nodes i and j , either i precedes j or *vice versa* only if both nodes are part of the tour. If node i is not selected to be part of the tour, then $v_{ij} = v_{ji} = 0$, $\forall j \in V \setminus \{r, i\}$. (In the context of the asymmetric TSP, we can impose that either i precedes j or *vice versa* for every pair of nodes, i.e., $v_{ij} + v_{ji} = 1$, $\forall i, j \in V \setminus \{r\}, i \neq j$.) Constraints (12) (resp., (13)) ensure that the first (resp., last) node after (resp., before) the root in the tour precedes (resp., is preceded by) all other nodes in the tour. The term $1 - y_j$ makes constraints (12) and (13) satisfied if node j is not selected to be in the tour. Constraints (14) are a lifted version of the three-city subtour elimination constraint $v_{ij} + v_{jk} + v_{ki} \leq 2$; see Sarin et al. (2005).

Sherali et al. (2006) applying the first-order RLT derived constraints (15), (16), and (20):

$$\sum_{i \in V \setminus \{r, k, j\}} t_{ij}^k + x_{rk} = v_{kj} \quad \forall k, j \in V \setminus \{r\}, i \neq j. \quad (20)$$

In the context of PCTSP constraints (20) must be modified, since they are not satisfied for pairs of nodes such that k is the first node after the root, that is, $x_{rk} = 1$, and j is not selected to make part of the tour, that is, $v_{kj} = 0$. (In the asymmetric TSP, if $x_{rk} = 1$ we have necessarily $v_{kj} = 1, \forall j \in V \setminus \{r, k\}$.) Constraints (17) and (18) replace (20) in the formulation for the PCTSP. Both constraints are satisfied if $v_{kj} = 0$: if $y_j = 0$ and $x_{rk} = 1$, constraints (15) imply $t_{ij}^k = 0, \forall i \in V \setminus \{r, k, j\}$; if $y_j = 1$ then $x_{rk} = t_{ij}^k = 0, \forall i \in V \setminus \{r, k, j\}$, since j precedes k in the tour or k not in the tour. Otherwise, $v_{kj} = 1$ implies $y_j = 1$, and (17) and (18) yield (20).

Although in the computational experiments reported by Sherali et al. (2006), ATSP6 consistently generated the tightest lower bounds among formulations analyzed in that paper, preliminary computational results have shown that formulation OPB' provides weak linear relaxation bounds for the PCTSP due to the substitution of (20) with (17) and (18). Thus, we propose an alternative adaptation of ATSP6 to the PCTSP preserving constraints (20). To do this we remove the term $1 - y_j$ of constraints (12) and let $v_{kj} = 1$ for every node j , even if j is not part of the tour, given that k is the first node after the root (constraints (22)). In this way, given a node k such that $x_{rk} = 1$, constraints (20) are satisfied even if $y_j = 0$ since now $v_{kj} = 1, \forall j \in V \setminus \{r, k\}$. Constraints (21) replace (11) to allow $v_{kj} = 1$ if $y_j = 0$, and constraints (23) correspond to (16) written as inequalities to be satisfied when j is not part of the tour. The resulting formulation, denoted by OPB, adds the following constraints to (2)–(6), (13)–(15), (19), and (20):

$$v_{ij} + v_{ji} \leq 1 \quad \forall i, j \in V \setminus \{r\}, i \neq j \quad (21)$$

$$v_{ij} \geq x_{ri} \quad \forall i, j \in V \setminus \{r\}, i \neq j \quad (22)$$

$$\sum_{k \in V \setminus \{r\}, i \neq j \neq k} t_{ij}^k + x_{ij} \leq v_{ij} \quad \forall i, j \in V \setminus \{r\}, i \neq j. \quad (23)$$

2.4. Enhanced circuit

Godinho et al. (2011) proposed an enhanced circuit formulation, denoted as EC-MCF++, which dominates the ordered path-based formulation due to Sherali et al. (2006) in terms of linear relaxation strength for the asymmetric TSP. The formulation uses an idea suggested by Godinho et al. (2008), where each node k belongs to a circuit. Godinho et al. (2010), see Godinho et al. (2014), further developed this idea modeling the asymmetric TSP on a two-layered hop-indexed graph, where the circuit is broken into two paths: the first layer represents the path from the root to node k and the second layer the returning path from node k to the root. The binary variable z_{ij}^h assumes 1 if (i, j) is the h th arc in the circuit starting at the root. For each node k , the binary variables $z1_{ij}^{hk}$ and $z2_{ij}^{hk}$ indicate whether the h th arc in the circuit is on the first layer or on the second layer, that is, $z1_{ij}^{hk} = 1$ (resp., $z2_{ij}^{hk}$) if (i, j) makes part of the path from the root to k (resp., from k to the root). The hop-indexed circuit formulation is enhanced using precedence variables $v_{ij}, i, j \in V \setminus \{r\}, i \neq j$. Let $|V| = n$. The formulation, denoted by EnC, adds the following constraints to (2)–(6) and (14):

$$\sum_{j \in V \setminus \{r\}} z1_{rj}^{1k} = y_k \quad \forall k \in V \setminus \{r\} \quad (24)$$

$$\sum_{j \in V \setminus \{r, i\}} z1_{ij}^{h+1, k} - \sum_{j \in V \setminus \{k, i\}} z1_{ji}^{hk} = 0 \quad \forall k, i \in V \setminus \{r\}, k \neq i, h = 1, \dots, n-2 \quad (25)$$

$$\sum_{j \in V \setminus \{k\}} z2_{kj}^{h+1, k} - \sum_{j \in V \setminus \{k\}} z1_{jk}^{hk} = 0 \quad \forall k \in V \setminus \{r\}, h = 1, \dots, n-1 \quad (26)$$

$$\sum_{j \in V \setminus \{k, i\}} z2_{ij}^{h+1, k} - \sum_{j \in V \setminus \{r, i\}} z2_{ji}^{hk} = 0 \quad \forall k, i \in V \setminus \{r\}, k \neq i, h = 2, \dots, n-1 \quad (27)$$

$$\sum_{h=1}^{n-2} \sum_{i \in V \setminus \{k, j\}} z1_{ij}^{hk} = v_{jk} \quad \forall j, k \in V \setminus \{r\}, j \neq k \quad (28)$$

$$z1_{ij}^{hk} + z2_{ij}^{hk} = z_{ij}^h \quad \forall k \in V \setminus \{r\}, \forall (i, j) \in A, h = 1, \dots, n \quad (29)$$

$$\sum_{h=1}^n z_{ij}^h = x_{ij} \quad \forall (i, j) \in A \quad (30)$$

$$v_{ij} + v_{ji} \leq y_j \quad i, j \in V \setminus \{r\}, j \neq i \quad (31)$$

$$z1_{ij}^{hk}, z2_{ij}^{hk} \in \{0, 1\} \quad \forall (i, j) \in A, \forall k \in V \setminus \{r\}, h = 1, \dots, n \quad (32)$$

$$z_{ij}^h \in \{0, 1\} \quad \forall (i, j) \in A, h = 1, \dots, n \quad (33)$$

$$v_{ij} \in \{0, 1\} \quad \forall i, j, k \in V \setminus \{r\}, k \neq j \neq i. \quad (34)$$

Formulation EC-MCF++ can be easily adapted to the PCTSP, it needs just to ensure that there is a circuit passing through node k if and only if k is part of the tour; see constraints (24). Thus, given $i, j \in V \setminus \{r\}, j \neq i$, exactly one of the variables v_{ij} or v_{ji} assume 1 only if i and j are part of the tour, otherwise both variables assume 0, see constraints (31). The remaining constraints are written as originally proposed by Godinho et al. (2011). Constraints (25)–(27) build the paths associated with each layer of the circuit passing through a visited node k using the hop index. Constraints (28) link the hop-indexed variables with the precedence variables. Constraints (29) ensure that an arc (i, j) is in the same position in the circuit of every visited node k , either in the first or in the second layer. Constraints (30) link the hop-indexed variables with the original arc variables.

2.5. Enhanced path-node

We propose a formulation using variables $z_{ij}^k, \forall i, j, k \in V \setminus \{r\}, k \neq i$. Given a triple of nodes different one another, $z_{ij}^k, i, j, k \in V, i \neq k \neq j$, assumes 1 if and only if all three nodes are visited, and the arc (i, j) is part of the tour in the path from the root to node k . Variables z_{ik}^k and $z_{ii}^k, i \neq k$, instead, are defined as follows:

- z_{ik}^k assumes 1 if and only if node i is the first node after the root in the tour and node k is not visited;

- z_{ii}^k assumes 1 if and only if node i is visited but is not the first node after the root in the tour and node k either precedes i in the tour or is not visited.

The formulation, denoted by EPN, adds the following constraints to (2)–(6):

$$\sum_{i \in V \setminus \{r, k\}} z_{ij}^k + x_{rj} = y_j \quad \forall k, j \in V \setminus \{r\}, j \neq k \quad (35)$$

$$\sum_{j \in V \setminus \{r\}} z_{ij}^k + x_{ik} = y_i \quad \forall i, k \in V \setminus \{r\}, i \neq k \quad (36)$$

$$z_{ij}^k \leq x_{ij} \quad \forall i, j, k \in V \setminus \{r\}, k \neq j \neq i \quad (37)$$

$$x_{ir} + x_{ri} \leq y_i \quad \forall i \in V \setminus \{r\} \quad (38)$$

$$z_{ij}^k \in \{0, 1\} \quad \forall i, j, k \in V \setminus \{r\}, k \neq i. \quad (39)$$

Let us consider initially in constraints (35) the case in which node j is visited, $y_j = 1$, but is not the first node after the root. We have $z_{jj}^k = 1$ if node k either precedes j in the tour or is not visited, otherwise there is an arc (i, j) in the path from the root to node k , and $z_{ij}^k = 1$. If node j is visited and it is the first node after the root, $x_{rj} = 1$, we have $z_{ij}^k = 0, \forall i, k \in V \setminus \{r\}, j \neq k, i \neq k$ (i.e., $z_{jj}^k = 0$). If node j is not visited, both sides are zero.

In an analogous manner, let us consider initially in constraints (36) the case in which node i is visited, $y_i = 1$, but is not the first node after the root. We have $z_{ii}^k = 1$ if node k either precedes i in the tour or is not visited, otherwise either arc (i, k) makes part of the tour or there is an arc (i, j) in the path from the root to node k . The last case applies if node i is the first node after the root and node k is visited. If node i is the first node after the root and node k is not visited, then we have $z_{ik}^k = 1$. If node i is not visited, both sides are zero. Constraints (37), for $j \neq i \neq k$, set to zero variables z_{ij}^k when arc (i, j) does not make part of the tour. Preliminary computational results have shown that constraints (38) may be useful to strengthen the linear relaxation bound provided by the formulation.

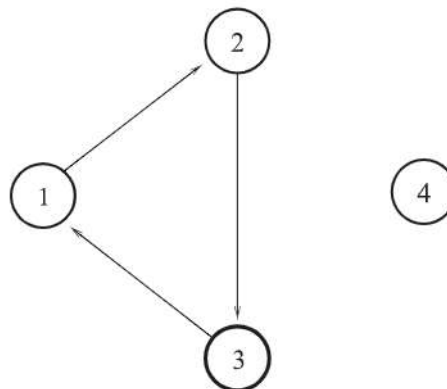
Figure 1 illustrates the use of variables z_{ij}^k with a feasible solution for an instance with 4 nodes where $r = 1$. In this solution, the only variables z_{ij}^k assuming 1 are the following: z_{24}^4 , since node 2 is the first node after the root in the tour and node 4 is not visited; z_{33}^2 , since node 3 is visited but is not the first node after the root in the tour and node 2 precedes 3, and z_{33}^4 , since node 3 is visited but is not the first node after the root in the tour and node 4 is not visited.

2.6. Prize-single commodity

The single-commodity formulation is known to provide a weak linear relaxation bound for the asymmetric TSP (see, for instance, Oncan et al., 2009). However, because the resulting linear programming problems are solved much faster and do not require too much memory, such a formulation can be attractive when attempting to get optimum solutions for the PCTSP using a MIP solver.

© 2021 The Authors.

International Transactions in Operational Research © 2021 International Federation of Operational Research Societies

Fig. 1. Illustrative example with root node $r = 1$.

Thus, we propose a single-commodity formulation where the flow along the tour accumulates the total prize collected as nodes are visited. The formulation, denoted by PSC, adds the following constraints to (2), (3), (5), and (6):

$$\sum_{j \in V \setminus \{i\}} (f_{ij} - p_i x_{ij}) - \sum_{j \in V \setminus \{r, i\}} f_{ji} = 0 \quad \forall i \in V \setminus \{r\} \quad (40)$$

$$\sum_{i \in V \setminus \{r\}} f_{ir} \geq g \quad (41)$$

$$f_{ij} \leq \bar{g} x_{ij} \quad \forall i, j \in V, i \neq r, i \neq j \quad (42)$$

$$f_{ij} \geq 0 \quad \forall i, j \in V, i \neq r, i \neq j. \quad (43)$$

In constraints (40), the flow leaving a visited node is increased by its prize. Constraint (41) ensures that a total flow of at least g arrives at the root node. Constraints (42) link the flow variables with the original arc variables, where $\bar{g} = \sum_{i \in V \setminus \{r\}} p_i$.

3. Lagrangian relaxation approach

As it will be seen in the computational results reported in Section 4, formulation EPN provided the best linear relaxation bounds among all formulations presented in Section 2. However, the resulting linear programming problem is quite often computationally expensive. Indeed, for nearly one third of the instances tested the linear programming relaxation could not be solved within three hours of computation. Thus, we propose a Lagrangian relaxation approach based on formulation EPN with the purpose of (i) getting tight lower bounds, possibly better than the linear relaxation by solving a knapsack problem as Lagrangian subproblem; (ii) finding optimal or near-optimal primal solutions through a heuristic scheme that exploits information of the Lagrangian subproblem; and (iii) bounding the optima within small optimality gaps with reasonable computational effort.

Table 1
Association of Lagrangian multipliers with constraints of EPN

Multipier	Constraints
$\pi_i \in \mathbb{R}, \forall i \in V$	(2)
$\phi_j \in \mathbb{R}, \forall j \in V$	(3)
$\lambda_{jk} \in \mathbb{R}, \forall j, k \in V \setminus \{r\}, j \neq k$	(35)
$\mu_{ik} \in \mathbb{R}, \forall i, k \in V \setminus \{r\}, i \neq k$	(36)
$v_{ij}^k \in \mathbb{R}_+, \forall i, j, k \in V \setminus \{r\}, i \neq j \neq k$	(37)
$\rho_i \in \mathbb{R}_+, \forall i \in V \setminus \{r\}$	(38)

Table 1 shows the association of Lagrangian multipliers with the constraints of EPN. Constraints (4)–(6) and (39) are left to the Lagrangian subproblem. With this choice of the constraints of EPN to be dualized, the Lagrangian subproblem decomposes into three independent subproblems LR^y , LR^x , and LR^z , associated with variables y , x , and z , respectively. The subproblems in the variables x and z are easily solved by inspection, while the resulting subproblem in the variable y is a knapsack problem. We hope in this way to possibly get lower bounds that are better than the linear relaxation bound. The term $\pi_r + \phi_r + \sum_{i \in V} w_i$ is constant since y_r is set to 1.

The subproblem LR^y consists of the following objective function subject to constraints (4) and (6):

$$\min \sum_{i \in V \setminus \{r\}} \left(\pi_i + \phi_i - \rho_i - w_i + \sum_{k \in V \setminus \{r, i\}} (\lambda_{ik} + \mu_{ik}) \right) y_i. \quad (44)$$

Let $\bar{y}_i = 1 - y_i, i \in V \setminus \{r\}$, $b = \sum_{i \in V \setminus \{r\}} p_i - g$, and $\alpha_i = \pi_i + \phi_i - \rho_i - w_i + \sum_{k \in V \setminus \{r, i\}} (\lambda_{ik} + \mu_{ik})$, then LR^y is equivalent to solving the knapsack problem plus the constant $\sum_{i \in V \setminus \{r\}} \alpha_i$:

$$\max \sum_{i \in V \setminus \{r\}} \alpha_i \bar{y}_i \quad (45)$$

$$\sum_{i \in V \setminus \{r\}} p_i \bar{y}_i \leq b \quad (46)$$

$$\bar{y}_i \in \{0, 1\} \quad \forall i \in V \setminus \{r\}. \quad (47)$$

The resulting knapsack problem can be solved using dynamic programming algorithms; see Toth (1980).

The subproblem LR^x decomposes for each arc $(i, j) \in A$ subject to constraints (5). Thus, it can be solved by inspection since x_{ij} assumes 1 if the reduced cost is negative, and 0 otherwise. The reduced cost of an arc depends on whether it is an ingoing arc to or an outgoing arc from the root or just an arc between two nodes in $V \setminus \{r\}$ as follows:

- for an arc (i, r)

$$c_{ir} + \rho_i - \pi_i - \phi_r; \quad (48)$$

- for an arc (r, j)

$$c_{rj} + \rho_j - \pi_r - \phi_j - \sum_{k \in V \setminus \{r, j\}} \lambda_{jk}; \quad (49)$$

- for an arc (i, j)

$$c_{ij} - \pi_i - \phi_j - \mu_{ij} - \sum_{k \in V \setminus \{r, i, j\}} v_{ij}^k. \quad (50)$$

Analogously, the subproblem LR^z decomposes for each triple of nodes subject to (39), and can be solved by inspection. The reduced costs are given by:

- associated with a variable $z_{ij}^k, i, j, k \in V \setminus \{r\}, i \neq j \neq k$:

$$v_{ij}^k - \lambda_{jk} - \mu_{ik}; \quad (51)$$

- associated with a variable $z_{ii}^k, i, k \in V \setminus \{r\}, i \neq k$:

$$-\lambda_{ik} - \mu_{ik}; \quad (52)$$

- associated with a variable $z_{ik}^k, i, k \in V \setminus \{r\}, i \neq k$

$$-\mu_{ik}. \quad (53)$$

To update the Lagrangian multipliers and solve the Lagrangian dual, we use the volume algorithm proposed by Barahona and Anbil (2000) with the implementation provided by the COIN-OR project <https://projects.coin-or.org/Vol>. See Shibasaki et al. (2021) for a comparison between volume and bundle methods on large-scale multicommodity network design.

3.1. Upper bounds

We propose a heuristic scheme to obtain an upper bound for the PCTSP given a subset of nodes $Y \subseteq V, r \in Y$, such that $\sum_{i \in Y} p_i \geq g$. Our aim is to embed the heuristic scheme into the Lagrangian relaxation approach hoping that promising subsets of nodes can be produced during the procedure to solve the Lagrangian dual. The heuristic takes the nodes in Y as the selected nodes to be visited, and finds an optimal tour. To do this, we apply the procedure proposed by Kumar and Li (2000) to transform an instance of the asymmetric TSP with $|Y|$ nodes into an instance of symmetric TSP with $2|Y|$ nodes, and then we use Concorde¹ available at <http://www.math.uwaterloo.ca/tsp/concorde.html>.

Then, we attempt to improve the current solution with a local search procedure composed of two phases. The first phase tries to improve the current solution by inserting nodes into the tour. A node $i \in V \setminus Y$ is inserted between nodes j and k belonging to the tour if $c_{ji} + c_{ik} < w_i + c_{jk}$,

¹Written by David Applegate, Robert E. Bixby, Václav Chvátal, and William J. Cook.

in this case we set $Y = Y \cup \{i\}$. Nodes are inserted into the tour on the first improvement basis. Once there are no more improving moves with nodes in $V \setminus Y$, the second phase tries to improve the current solution by removing nodes from the tour. Let $j, i, k \in Y, i \neq r$, be a triple of successive nodes in the tour, and $s_i = w_i + c_{jk} - c_{ji} - c_{ik}$. The set of candidate nodes to be removed from the tour is given by $\bar{Y} = \{i \in Y \setminus \{r\} : s_i < 0 \text{ and } \sum_{u \in Y \setminus \{i\}} p_u \geq g\}$. The nodes to be actually removed from the tour are chosen by solving the following integer programming problem where variable $q_i, i \in \bar{Y}$, assumes 1 if node i is removed, and 0 otherwise:

$$\min \sum_{i \in \bar{Y}} s_i q_i \quad (54)$$

$$\sum_{i \in \bar{Y}} p_i q_i \leq \sum_{i \in \bar{Y}} p_i - g \quad (55)$$

$$q_i + q_j \leq 1 \quad \forall i, j \in \bar{Y} : \text{arc}(i, j) \text{ makes part of the tour} \quad (56)$$

$$q_i \in \{0, 1\} \quad \forall i \in \bar{Y}. \quad (57)$$

The objective function (54) accounts for the maximum possible improvement in the current solution. Constraints (55) ensure that the prize is still collected by the remaining nodes in the tour. Constraints (56) assert that no two successive nodes in the tour will be both removed. Let $\bar{Y}^*$ be the set of nodes for which $q_i = 1$ in an optimal solution to (54)–(57), then we use Concorde to find an optimal tour with the nodes in $Y \setminus \bar{Y}^*$.

3.2. Valid inequalities

The Lagrangian relaxation bound may be strengthened with the addition of cuts that are currently violated by the Lagrangian solution in a relax-and-cut fashion; see Guignard (2003) and Lucena (2005). Typically, inequalities from a family of exponentially many candidates are dualized as they become violated by the Lagrangian solution. In our Lagrangian approach, we try to identify cost-cover and cycle-cover inequalities as they were successfully used to strengthen lower bounds for the PCTSP by Fischetti and Toth (1988) and Bérubé et al. (2009).

The cost-cover inequalities are valid when arc costs satisfy the triangle inequality. Let UB be an upper bound on the optimal value of the PCTSP instance, and L_Y be a lower bound on the optimal value of a tour given a subset $Y \subseteq V, r \in Y$ (lower bound to a TSP instance on Y). If $L_Y > UB$, then the following inequality is valid for the PCTSP:

$$\sum_{i \in Y} y_i \leq |Y| - 1. \quad (58)$$

At each iteration when solving the Lagrangian dual problem, we check whether a violated cost-cover cut can be generated with the set Y of those nodes for which y_i is equal to 1 in the optimal solution of LR^y ; see (44)–(47). This implies solving a TSP instance with Concorde if set Y has not been checked in a previous iteration.

The cycle-cover inequalities exploit the fact that a feasible solution for the PCTSP is a tour through a subset of nodes collecting at least the total prize g . Thus, given a subset $Y \subseteq V$, $r \in Y$, such that $\sum_{i \in Y} p_i < g$, the following inequality is valid for the PCTSP:

$$\sum_{i,j \in Y: i \neq j} x_{ij} \leq \sum_{i \in Y} y_i - 1. \quad (59)$$

As observed by Bérubé et al. (2009), the separation routine of (7) based on the maximum flow problem can be used in an attempt to identify possible violated cycle-cover inequalities. Consider the graph $G = (\bar{V}, \bar{A})$ with sets $\bar{V}$ and $\bar{A}$ defined in Section 2.1. For each $v \in \bar{V}$, one simply needs to check for violation of a cycle-cover inequality if the summation of prizes in the set Y in the minimum-capacity cut containing node r is less than g . As with GSEC, we used the implementation due to Dinitz (2006) for the maximum flow problem, available at <https://gist.github.com/gabber12/5257878>, whose complexity is $O(n^2m)$ for a graph with n nodes and m arcs.

3.3. Algorithm overview

Algorithm 1 provides a pseudo-code of the proposed Lagrangian relaxation approach for the PCTSP. The initialization in Line 1 sets to 0 the iteration counter it and the best lower bound LB attained with the Lagrangian dual. The heuristic scheme of Section 3.1 is called in the beginning to obtain the first upper bound, and possibly within each block of h_{it} iterations in the main loop. In Line 2, a set Y serving as input to the heuristic is obtained in a greedy way as follows: Y initially contains r , then the nodes in $V \setminus \{r\}$ are sorted in decreasing order of w_i/p_i and inserted in Y until $\sum_{i \in Y} p_i \geq g$. The heuristic is applied on Y and the first upper bound is obtained in Line 3.

The main loop is performed while the stopping criterion is not met. We used the stopping criterion of the volume algorithm as implemented in the COIN-OR project <https://projects.coin-or.org/Vol>.² In Line 5, LB_{it} , which stores the best lower bound within a block of h_{it} iterations, is set to 0 for each new block of h_{it} iterations. If the lower bound LB' obtained by solving the Lagrangian subproblems in Line 6 improves the best lower bound found so far, then LB , LB_{it} , and Y are updated in Lines 9, 10, and 11, respectively. Otherwise, it is checked in Line 13 if LB' improves the best lower bound found within the block of h_{it} iterations, and, if it is the case, LB_{it} and Y are updated in Lines 14 and 15, respectively. The Lagrangian multipliers are updated in Line 16 according to the volume algorithm. We try to identify violated cost-cover and cycle-cover cuts only if the set Y' containing the nodes for which $y_i = 1$ is the optimal solution of LR^y has not been generated in a previous iteration (otherwise the cuts, if any, are already dualized in the objective functions of LR^y and LR^x). This is checked in Line 17, and in the affirmative case, Concorde is called to find the optimal tour with the nodes in Y' in Line 18. In Line 19, it is checked if the current upper bound can be updated by solving the TSP with nodes in Y' . The procedures to identify violated cost-cover

²Barahona and Ladanyi, an implementation of the volume algorithm, April 19, 2007. Let z be the value of the current dual solution, and p be the value of a current near-feasible primal solution according to a maximum primal violation for a given precision. There are three termination criteria: (i) the maximum number of iterations; (ii) if $|z| > 0.0001$ and $\frac{|z-p|}{|z|} < \gamma$, where γ is a given precision; and (iii) if $|z| \leq 0.0001$ and $|z-p| < \tau$, where τ is a given precision; see also Barahona and Anbil (2000).

Algorithm 1. Lagrangian relaxation approach for the PCTSP

```

1: Initialize Lagrangian multipliers, set  $it = 0$  and  $LB = 0$ 
2: Obtain a set  $Y \subseteq V$ ,  $r \in Y$ , such that  $\sum_{i \in Y} p_i \geq g$ 
3: Apply the heuristic of Section 3.1 on  $Y$  to obtain an initial feasible solution and compute  $UB$ 
4: while not stopping criterion do
5:   if  $it \bmod h_{it} = 0$  then  $LB_{it} = 0$ 
6:   Solve the Lagrangian subproblems  $LR^y$ ,  $LR^x$ , and  $LR^z$  and obtain a lower bound  $LB'$ 
7:    $\triangleright$  Let  $Y'$  and  $X'$  be the sets of variables equal to 1 in subproblems  $LR^y$  and  $LR^x$ , respectively
8:   if  $LB' > LB$  then
9:      $LB = LB'$ 
10:     $LB_{it} = LB'$ 
11:     $Y = Y'$ 
12:   else
13:     if  $LB' > LB_{it}$  then
14:        $LB_{it} = LB'$ 
15:        $Y = Y'$ 
16:   Update the Lagrangian multipliers
17:   if set  $Y'$  has not been generated before then
18:     Find an optimal tour with the nodes in  $Y'$  and compute  $UB'$ 
19:     if  $UB' < UB$  then  $UB = UB'$ 
20:     Check whether a violated cost-cover cut can be generated with the set  $Y'$ 
21:     Check whether a violated cycle-cover cut can be generated with the set  $Y'$ 
22:    $it = it + 1$ 
23:   if  $it \bmod h_{it} = 0$  and set  $Y$  has not been generated before then
24:     Apply the heuristic of Section 3.1 on  $Y$  to obtain a feasible solution and compute  $UB'$ 
25:     if  $UB' < UB$  then  $UB = UB'$ 
26: end-while

```

(if arc costs satisfy the triangle inequality) and cycle-cover cuts are called in Lines 20 and 21, respectively. An identified cut is dualized in the objective functions of LR^y and LR^x , accordingly, and is kept until the end of the algorithm. In Line 22, the iteration counter is increased. In Line 23, it is checked at the end of a block of h_{it} iterations if the set Y of nodes leading the best lower bound within the block has not been generated in a previous iteration. In the affirmative case, the heuristic proposed in Section 3.1 is applied in Line 24, and the upper bound is possibly updated in Line 25.

4. Computational results

Our experiments were performed on a total of 252 instances generated from benchmark asymmetric TSP instances. They are divided into three groups: *realworld*, *crane*, and *stilt*. Cirasella et al. (2001) gathered instances from the literature and introduced a variety of new random instances. They denominated by *realworld* the asymmetric TSP instances available at the TSPLIB and at other sources. We used all the graphs with less than 500 nodes from this group *realworld* to generate instances for the PCTSP. This include the 4 *atex* graphs with 16, 32, 48, and 72 nodes; the 2 *code* graphs with 198 and 253 nodes; the 5 *dc* graphs with 112, 126, 134, 176, and 188 nodes; the 2 *td* graphs with 101 and 317 nodes; and all the 19 graphs of the TSPLIB

© 2021 The Authors.

International Transactions in Operational Research © 2021 International Federation of Operational Research Societies

which are graphs up to 443 nodes. The TSPLIB contains 9 graphs denoted by *ftv*, 8 of them up to 71 nodes, and 1 of them, *ftv170.atsp*, with 171 nodes. Fischetti and Toth (1997) generated other 8 graphs considering the first 91 nodes, 101 nodes, and so on until 161 nodes of instance *ftv170.atsp*. Cirasella et al. (2001) provide details from which applications these *realworld* instances for the asymmetric TSP come from. All these graphs are available either at the website <http://elib.zib.de/pub/mp-testdata/tsp/tsplib/atsp/index.html> or at the website <https://sites.google.com/site/atspinstances/cirasella-instances>. Besides, we also consider one more graph with the first 81 nodes of instance *ftv170.atsp*. Thus, a total of 41 graphs from this group *realworld* were used to generate 123 instances for the PCTSP. The penalty w_i and the prize p_i , $i \in V \setminus \{r\}$, are integers drawn at random from the uniform distribution in the range (1,100). For each graph, three PCTSP instances were generated by setting the total prize to be collected as $g = \alpha \sum_{i \in V \setminus \{r\}} p_i$, $\alpha \in \{0.2, 0.5, 0.8\}$, in a similar way to what was done in Bérubé et al. (2009).

We also used the 23 graphs of group *crane* and the 20 graphs of group *stilt* introduced by Cirasella et al. (2001) to generate instances for the PCTSP. The group *crane* contains 3 graphs with 66 nodes, 10 with 100 nodes, and 10 with 316 nodes. The group *stilt* contains 10 graphs with 100 nodes, and 10 with 316 nodes. We chose such graphs because in the experiments reported by Cirasella et al. (2001), these groups presented the largest optimality gaps with respect to the Held–Karp lower bound (Held and Karp, 1970, 1971) for the 100 nodes asymmetric TSP instances. Instances from group *crane* come from the stacker crane problem, where a crane must pick up and deliver objects between given locations. Instances from group *stilt* come from industrial operations performed by drilling machines. For each graph, 3 PCTSP instances were generated by setting the total prize to be collected as $g = \alpha \sum_{i \in V \setminus \{r\}} p_i$, $\alpha \in \{0.2, 0.5, 0.8\}$, resulting in a total of 129 instances obtained from these groups. The prize p_i , $i \in V \setminus \{r\}$, is also drawn at random from the uniform distribution in the range (1,100), but the penalty w_i , $i \in V \setminus \{r\}$, is drawn in the range (1, 10^5). We used a larger range for the penalty values because the arc costs in these graphs have a magnitude of 10^6 .

The experiments were conducted on a PC Intel Core i7-2600, 3.40 GHz, 16 GB of RAM, running Linux 64 bits. The optimization software Gurobi version 7.5.2 was used to solve linear relaxation and mixed integer problems. The linear relaxation values were obtained at the root node running Gurobi with the presolve and cuts disallowed. The MIP problems were run with the Gurobi standard setting. We were able to obtain optimal values for all the 123 *realworld* instances with formulation PSC, c.f., Section 2.6, running Gurobi without time limit.

4.1. Linear relaxation

4.1.1. Group *realworld* instances

Table 2 shows results on the *realworld* instances comparing the linear relaxations of the six formulations presented in Section 2 in terms of the tightness of their lower bounds and the computational effort required to get them. The linear relaxation problems were run considering a time limit of 10,800 seconds. In the upper part of Table 2, for each formulation, we report in the first line the number of instances for which the linear relaxation problem was solved within the time limit. All the six formulations were able to obtain linear relaxation bounds for 44 of the 123 tested instances. Therefore, the remaining lines in Table 2 show the results for these 44 instances. The second line

Table 2

Results comparing the linear relaxations of the six formulations considering the time limit of 10,800 seconds

		Formulations					
		GSEC Section 2.1	MCF Section 2.2	OPB Section 2.3	EnC Section 2.4	EPN Section 2.5	PSC Section 2.6
#SOLVED _{LP}		123	77	57	44	78	123
AVG_gap		10.93	1.77	1.77	1.76	1.48	11.31
AVG_time		2	22	131	2716	45	2
Instance							
$\alpha = 0.2$							
ft70	gap	3.93	1.62	1.62	1.62	1.42	1.74
	time	47	24	172	8086	55	22
ft53	gap	4.89	0.68	0.66	0.65	0.65	13.63
	time	2	17	75	947	18	1
ry48p	gap	19.60	10.61	10.61	9.93	8.67	18.26
	time	1	25	116	703	35	1
atex1	gap	8.96	8.96	8.96	8.96	0.00	8.96
	time	0	0	0	38	0	0
atex4	gap	21.72	0.42	0.42	0.42	0.00	21.70
	time	0	21	191	8247	17	0
$\alpha = 0.5$							
atex1	gap	25.65	22.38	22.38	22.38	21.00	32.05
	time	0	0	0	45	0	0
atex4	gap	20.29	0.02	0.02	0.02	0.00	19.90
	time	0	17	162	8923	16	1
$\alpha = 0.8$							
atex1	gap	34.93	2.73	2.73	2.73	2.58	22.91
	time	0	0	0	62	0	0

presents the average optimality gaps, computed as $(Opt - LB)/Opt$ in percentage, where Opt is the optimal value and LB the lower bound. The third line shows the average computational times in seconds. The formulations MCF, OPB, EnC, and EPN provided the same linear relaxation bounds for 36 of the 44 instances. Detailed results for the eight dissonant instances are shown at the bottom of Table 2. The best gaps are highlighted in bold. The formulations GSEC and PSC obtained the weakest bounds in all tested instances.

The proposed formulation EPN obtained the strongest linear relaxation bounds whenever it was able to generate these bounds within the time limit. In addition, the computational time to obtain these bounds was considerably less than that spent by the second-best formulations OPB and EnC. On the other hand, the linear relaxation problems derived from formulation PSC were solved much faster and using far less memory than the other formulations. Consequently, only with PSC we were able to obtain the optimal mixed integer values for all 123 realworld instances running Gurobi without time limit.

Table 3 compares the performances of formulations GSEC, MCF, EPN, and PSC on the 39 instances in addition to the 44 detailed in Table 2 for which either MCF or EPN solved the linear relaxation problem within the time limit. The upper part of Table 3 shows average results for the

Table 3

Results on the 39 instances in addition to the 44 detailed in Table 2 for which either MCF or EPN solved the linear relaxation within the time limit of 10,800 seconds

		Formulations			
		GSEC Section 2.1	MCF Section 2.2	EPN Section 2.5	PSC Section 2.6
AVG_gap		12.45	0.83	0.82	10.40
AVG_time		5	1234	1025	11
Instance					
$\alpha = 0.2$					
krol124	gap	24.40	3.09	2.84	15.69
	time	25	4993	1968	16
ftv120	gap	24.29	20.78	–	21.67
	time	31	3164	–	36
ftv130	gap	11.72	7.57	–	9.55
	time	10	9378	–	45
dc126	gap	0.18	–	0.05	0.11
	time	1	–	481	5
dc134	gap	0.22	–	0.03	0.12
	time	63	–	4523	41
$\alpha = 0.5$					
ftv120	gap	24.29	20.78	–	21.67
	time	29	5212	–	36
ftv130	gap	11.72	7.57	–	9.55
	time	38	10709	–	45
dc126	gap	2.18	–	0.50	1.05
	time	2	–	3247	12
dc134	gap	0.22	–	0.03	0.12
	time	72	–	4678	44
$\alpha = 0.8$					
ftv120	gap	24.29	20.78	–	21.67
	time	32	3912	–	36
dc126	gap	5.08	–	0.17	3.20
	time	12	–	4797	28
dc134	gap	0.53	–	0.10	0.30
	time	94	–	5303	76

28 of the 39 instances analyzed in this table for which both MCF and EPN solved the linear relaxation problem. Formulations MCF and EPN obtained the best linear relaxation bounds at the highest computational costs when compared to PSC and GSEC. More specifically, MCF and EPN provided the same bounds for all these 28 instances, except for instance krol124 with $\alpha = 0.2$, for which detailed results along with instances solved by only one formulation are shown at the bottom of the Table 3. A symbol “–” indicates that a formulation did not solve the linear relaxation problem within the time limit. Formulation OPB does not appear in Table 3 because it was able to solve the linear relaxation problem within the time limit only for 13 of the 39 instances represented in Table 3, providing the same bounds obtained by MCF and EPN but requiring much higher computational time.

Table 4

Results on the 69 crane and stilt instances for which formulations GSEC, MCF, EPN, and PSC solved the linear relaxation problem within the time limit of 10,800 seconds

		Formulations			
		GSEC Section 2.1	MCF Section 2.2	EPN Section 2.5	PSC Section 2.6
#SOLVED _{LP}		129	69	69	129
AVG_gap		12.31	1.77	1.65	11.11
AVG_time		38	557	676	21
Instance					
$\alpha = 0.2$					
crane66_2	gap	8.26	1.73	1.42	8.34
	time	1	71	45	2
crane100_2	gap	9.62	1.81	1.59	11.92
	time	25	712	836	20
crane100_3	gap	8.39	1.11	0.42	8.13
	time	23	569	824	30
crane100_4	gap	10.05	0.15	0.11	8.09
	time	22	711	858	22
crane100_8	gap	8.78	0.52	0.38	8.2
	time	25	621	753	21
stilt100_3	gap	12.2	9.88	8.66	19.8
	time	34	636	796	25
stilt100_6	gap	13.4	8.20	7.84	19.6
	time	34	761	878	24
stilt100_7	gap	14.1	6.50	5.25	18.2
	time	36	744	851	22
stilt100_8	gap	13.9	6.97	5.21	13.4
	time	33	723	881	21
stilt100_9	gap	15.4	3.77	2.28	13.9
	time	40	596	811	28
Instance					
$\alpha = 0.5$					
crane100_3	gap	8.04	0.27	0.26	7.75
	time	24	609	731	42
stilt100_3	gap	19.9	1.98	1.64	17.6
	time	36	519	739	17
stilt100_7	gap	15.5	1.15	1.12	18.7
	time	47	694	773	24
stilt100_8	gap	21.4	1.21	0.98	12.0
	time	29	655	792	29

4.1.2. Group crane and group stilt instances

Table 4 shows results comparing formulations GSEC, MCF, EPN, and PSC on the crane and the stilt instances. All the four formulations were able to obtain linear relaxation bounds for 69 of the 129 tested instances considering a time limit of 10,800 seconds. Formulation EnC did not solve the linear relaxation problem for any of the instances in these groups within the time limit,

© 2021 The Authors.

International Transactions in Operational Research © 2021 International Federation of Operational Research Societies

and OPB solved only six of them providing the same lower bounds obtained by MCF. Table 4 has the same structure as Table 3. As observed with the *realworld* instances, EPN always provided lower bounds at least as good as the ones provided by MCF. Moreover, EPN provided better lower bounds for 14 of the *crane* and *stilt* instances, as detailed at the bottom part of Table 4.

4.2. Lagrangian relaxation approach

We now report detailed results obtained with the proposed Lagrangian approach presented in Algorithm 1, c.f., Section 3.3. We ran Algorithm 1 until the stopping criterion of the volume algorithm in the implementation provided by the COIN-OR project <https://projects.coin-or.org/Vol> is met, or $UB - LB < 1$, given that arc costs and penalties are all integer in the instances used in this computational experiment. The heuristic scheme was called if a new set Y was obtained within a block of $h_{it} = 100$ iterations.

The volume algorithm has parameters that control its behavior. These parameters directly influence the results and the convergence of the algorithm. Hence, a tuning phase is required. The works of Barahona and Anbil (2000), Bahiense et al. (2002), and Frangioni et al. (2017) were used to define the initial set of possible values for the parameters. This phase required substantial computational experimentation since several combinations were tested. We first selected a subset of instances up to 80 nodes from the group *realworld* to test a very large number of combinations of values. For instance, parameter *greentestinv1* assumed 1, 2, 4, 8, 10; parameter *yellowtestinv1* assumed 2, 4, 6, 8, 10, 20, 30, 40, 50, 80, 100, 150, 200, 300, 400, 500; and parameter *redtestinv1* assumed 2, 4, 6, 8, 10, 20, 30, 40, 50, 80, 100, 150, 200, 300, 400, 500. We observed that the best results were always obtained with parameter *greentestinv1* set to 1. Then, with *greentestinv1* set to 1, we reduced the number of possible combinations and increased the subset of instances to refine the tuning. In this second round, we observed that there was no change when setting parameter *yellowtestinv1* greater than 10, and parameter *redtestinv1* greater than 20. This process was repeated until a satisfactory behavior was found for all the instances. For the sake of clarity and conciseness, the values used to obtain the results we report were the following:

- *greentestinv1* = 1; *lambdainit* = 0.1; *alphainit* = 0.1
- Group *realworld* instances
 - *yellowtestinv1* = 2
 - $|V| < 70$, except instances *ft53*, *p43*, and *ry48p*: *redtestinv1* = 4; *alphamin* = 0.0001; *alphafactor* = 0.9 (for *atex* instances *alphafactor* = 0.5); *alphaint* = 100
 - $|V| \geq 70$ and also instances *ft53*, *p43*, and *ry48p*: *redtestinv1* = 10; *alphamin* = 0.00001; *alphafactor* = 0.03; *alphaint* = 150
- Group *crane* instances
 - *yellowtestinv1* = 10; *redtestinv1* = 10; *alphamin* = 0.00001; *alphafactor* = 0.5; *alphaint* = 100
- group *stilt* instances
 - *yellowtestinv1* = 10; *redtestinv1* = 10; *alphamin* = 0.00001; *alphafactor* = 0.5; *alphaint* = 150

4.2.1. Group realworld instances

The graphs br17, kro124, ry48p, p43, rbg323, rbg358, rbg403, rbg443, code198, code253, dc126, td100_1, and td31_10 do not satisfy the triangle inequality, so the cost-cover cuts were not applied to the instances derived from these graphs. Table 5 shows the computational results for the 78 realworld instances for which linear relaxations were solved by EPN within the time limit of 10,800 seconds. The first column identifies the instance. The second column (gap UB) presents the percentage gap $(UB - Opt)/Opt$ between the best upper bound UB found by Algorithm 1 and the optimum value Opt . The third column (gap LB) exhibits the percentage gap $(Opt - LB)/Opt$ between the lower bound LB returned by the Lagrangian relaxation and the optimum value Opt . The fourth column (gap LP) presents the percentage gap $(LP - LB)/LP$ between the lower bound LB returned by the Lagrangian relaxation and the lower bound LP provided by the linear relaxation of EPN, thus a negative value in this column means that the Lagrangian bound is better than the linear relaxation bound. The fifth column (gap Δ) presents the percentage gap $(UB - LB)/UB$ between the best upper bound UB found by Algorithm 1 and the lower bound LB returned by the Lagrangian relaxation, that is, the optimality gap computed exclusively with the bounds provided by Algorithm 1. The sixth column exhibits the number of iterations Algorithm 1 executed to reach the stop criterion. The seventh column shows the total computational time in seconds spent by Algorithm 1, and the eighth column shows the time in seconds to obtain the optimal mixed integer value with PSC running Gurobi without time limit.

The heuristic scheme embedded in Algorithm 1 succeeded in obtaining the optimal values for 43 of 78 instances for which linear relaxations were solved by EPN within the time limit of 10,800 seconds. These results are highlighted in bold. Moreover, the heuristic was efficient to obtain near-optimal solutions when the optimal was not found. In fact, gap UB was no greater than 1% for 74% of the instances (58 out of 78 instances), and its maximum value was 2.28%. The Lagrangian relaxation provided in most cases tighter lower bounds, presenting on average 1.15% of optimality gap. The lower bounds equaled optimal values for 32 of 78 instances, gap LB was no greater than 1% for 58% of the instances (45 out of 78 instances), and no greater than 2% for 85% of the instances (66 out of 78 instances). The gap with respect to the linear relaxation bound was 0.06% on average. The bounds obtained by the Lagrangian relaxation improved upon the linear relaxation bounds for 23 of 78 instances, even though only for 3 instances the improvements were larger than 1%. Algorithm 1 returned in most cases small optimality gaps, 1.54% on average. Algorithm 1 closed the optimality gap for 31 of 78 instances, gap Δ was no greater than 1% for 56% of the instances (44 out of 78 instances), and no greater than 2% for 68% of the instances (53 out of 78 instances). Only for 4 of 78 instances, gap Δ was greater than 5%. PSC solved very fast most of the instances in Table 5. Nevertheless, Algorithm 1 obtained the optimal solutions faster than running PSC with Gurobi for the larger instances ftv80, ftv90, ftv100, ftv110, dc112, dc126, dc134, and td100_1 whatever the value of α , and also for atex4 and atex5 for $\alpha = 0.8$. We remark that the time spent by Algorithm 1 was not directly proportional to the number of iterations executed. For example, Algorithm 1 spent 306 seconds to execute 4429 iterations on instance dc112 $\alpha = 0.2$, and 551 seconds to execute 1169 iterations on instance dc126 $\alpha = 0.2$. This is because in some instances the knapsack subproblem returns quite often the same set of nodes, and then Concorde and the heuristic routine are not called so often, saving computational time.

Table 6 shows results for the remaining 45 realworld instances for which EPN was not able to solve the linear relaxation problem within the time limit of 10,800 seconds. Therefore, it has the

Table 5

Results obtained by Algorithm 1 for the 78 *realworld* instances for which EPN linear relaxation values are available

Instance	gap <i>UB</i>	gap <i>LB</i>	gap <i>LP</i>	gap Δ	Iterations	Time Algo 1	Time PSC (MIP)
$\alpha = 0.2$							
br17	0.00	0.00	0.00	0.00	4181	2	0
ft53	1.20	2.21	1.57	3.37	4633	105	46
ft70	1.34	1.27	0.71	2.57	2708	286	63
ftv33	0.00	0.00	0.00	0.00	5679	19	1
ftv35	0.00	0.00	−0.49	0.00	7157	57	1
ftv38	0.00	0.59	−0.21	0.59	2583	47	5
ftv44	0.00	0.08	0.08	0.08	7906	76	2
ftv47	0.00	0.00	0.00	0.00	5130	203	3
ftv55	0.00	1.74	0.65	1.74	3491	599	8
ftv64	0.00	0.26	0.26	0.26	10,456	316	14
ftv70	0.00	1.98	0.54	1.98	11,608	607	15
ftv80	0.00	0.00	0.00	0.00	4638	387	723
ftv90	0.00	0.00	0.00	0.00	5993	400	825
ftv100	0.00	0.00	0.00	0.00	4167	424	974
ftv110	0.00	0.00	−0.05	0.00	6057	665	2132
kro124	2.20	4.06	1.26	6.13	16,435	2498	136
p43	0.00	0.00	0.00	0.00	37,210	517	13
ry48p	0.09	8.04	−0.69	8.13	4985	81	14
atex1	0.00	0.00	0.00	0.00	1655	41	0
atex3	0.00	0.00	0.00	0.00	1312	34	1
atex4	0.00	0.00	0.00	0.00	4058	212	95
atex5	0.33	1.19	0.02	1.52	7674	113	17
dc112	0.00	0.00	0.00	0.00	4429	306	412
dc126	1.25	1.15	1.10	2.36	1169	551	707
dc134	0.00	0.00	−0.03	0.00	3632	678	1099
td100_1	1.24	1.65	1.01	2.85	2356	452	556
Average	0.30	0.93	0.22	1.21	6589	372	302
$\alpha = 0.5$							
br17	0.00	0.00	0.00	0.00	4181	1	0
ft53	1.47	2.97	1.28	4.38	7657	193	28
ft70	1.64	1.29	1.08	2.89	2708	148	38
ftv33	0.00	0.00	0.00	0.00	5270	40	1
ftv35	0.34	0.43	−0.06	0.77	7157	59	1
ftv38	0.00	0.59	−0.21	0.59	6395	52	3
ftv44	0.00	0.16	0.16	0.16	6652	83	2
ftv47	0.00	0.00	0.00	0.00	5852	223	3
ftv55	0.13	1.74	0.65	1.86	3580	394	9
ftv64	0.00	0.26	0.26	0.26	10,277	472	12
ftv70	0.24	2.28	0.84	2.51	8924	483	13
ftv80	0.29	0.07	0.07	0.36	6831	379	734
ftv90	0.00	0.00	0.00	0.00	5062	436	827
ftv100	0.00	0.00	0.00	0.00	4167	464	995
ftv110	0.00	0.00	−0.05	0.00	6057	813	2105
kro124	2.15	2.50	0.97	4.55	9228	1484	206
p43	0.00	0.00	0.00	0.00	38,524	581	17

Continued

© 2021 The Authors.

International Transactions in Operational Research © 2021 International Federation of Operational Research Societies

750

G. Pantuza and M.C. de Souza / Intl. Trans. in Op. Res. 29 (2022) 729–759

Table 5
Continued

Instance	gap UB	gap LB	gap LP	gap Δ	Iterations	Time Algo 1	Time PSC (MIP)
ry48p	0.34	2.95	0.00	3.28	6109	121	8
atex1	0.38	14.58	−8.13	14.90	1983	35	2
atex3	1.04	3.59	−0.21	4.58	2412	33	3
atex4	0.00	0.00	0.00	0.00	4867	392	120
atex5	1.05	0.44	0.02	1.47	7759	227	35
dc112	0.00	0.00	−0.46	0.00	4720	389	803
dc126	1.25	1.85	1.35	3.06	2138	693	1075
dc134	0.00	0.00	−0.03	0.00	4704	673	1284
td100_1	1.07	1.17	0.80	2.22	2761	492	630
Average	0.44	1.42	−0.06	1.84	6768	360	344
$\alpha = 0.8$							
br17	0.00	0.00	0.00	0.00	4181	1	1
ft53	1.25	1.39	0.96	2.60	1083	193	64
ft70	1.16	1.15	0.95	2.29	5466	57	56
ftv33	0.00	0.09	−1.30	0.09	5027	50	1
ftv35	0.00	0.67	−0.31	0.67	6570	73	2
ftv38	0.49	1.80	−0.02	2.29	6559	127	3
ftv44	0.00	1.60	0.02	1.60	6761	190	6
ftv47	0.07	1.45	0.00	1.52	9482	19	6
ftv55	0.06	1.74	0.65	1.80	5593	475	10
ftv64	0.13	0.00	−0.07	0.13	920	67	60
ftv70	0.00	0.87	−3.13	0.87	8548	397	101
ftv80	0.00	0.00	0.00	0.00	864	676	726
ftv90	0.00	0.00	0.00	0.00	3795	633	828
ftv100	0.00	0.00	0.00	0.00	5709	375	1026
ftv110	0.00	0.00	−0.05	0.00	6041	718	2295
kro124	2.28	2.13	1.15	4.31	772	217	537
p43	0.00	0.00	0.00	0.00	8992	118	6
ry48p	0.11	1.80	−0.16	1.90	675	19	13
atex1	0.50	1.87	−0.74	2.35	3014	63	1
atex3	1.12	1.08	−0.43	2.17	2566	58	50
atex4	1.33	4.39	−0.11	5.65	4317	317	412
atex5	1.15	3.13	0.02	4.23	8073	312	366
dc112	0.00	0.00	−0.27	0.00	5278	521	4484
dc126	1.27	1.73	1.56	2.97	2086	603	1127
dc134	0.39	0.17	0.07	0.56	5263	996	5578
td100_1	1.28	1.81	1.63	3.05	3291	576	988
Average	0.49	1.11	0.02	1.58	4651	302	721

same columns as Table 5 less than gap LP . Overall, Algorithm 1 obtained an optimality gap of 1.48% on average for these 45 larger instances.

Algorithm 1 was very efficient to deal with the ftv instances, returning solutions with gap UB no greater than 0.40% for all the 18 instances in this group, and finding optimal solutions for 7 of them. The Lagrangian relaxation provided tighter lower bounds for these instances, gap LB is no greater than 0.67% (in instance ftv150 $\alpha = 0.2$ the lower bound equals the optimal value, but the

© 2021 The Authors.

International Transactions in Operational Research © 2021 International Federation of Operational Research Societies

Table 6
Results obtained by Algorithm 1 for the 45 *realworld* instances for which EPN linear relaxation values are not available

Instance	gap <i>UB</i>	gap <i>LB</i>	gap Δ	Iterations	Time Algo 1	Time PSC (MIP)
$\alpha = 0.2$						
ftv120	0.00	0.10	0.10	7356	1422	2519
ftv130	0.05	0.05	0.09	7356	1162	3179
ftv140	0.09	0.09	0.18	7792	1738	8262
ftv150	0.04	0.00	0.04	6453	1672	9342
ftv160	0.00	0.32	0.32	7321	2269	8199
ftv170	0.00	0.41	0.41	7905	3263	10,164
rbg323	0.51	1.69	2.18	25,512	17,502	129,954
rbg358	0.11	0.30	0.40	30,215	16,752	132,963
rbg403	0.05	2.34	2.38	38,112	22,207	180,595
rbg443	0.42	1.31	1.71	37,034	29,642	183,949
dc176	1.21	0.36	1.54	5179	986	3408
dc188	0.82	0.86	1.67	7635	1692	3344
code198	2.32	1.78	4.01	8455	1961	3425
code253	2.46	1.68	4.04	9592	2593	4652
td316_10	0.56	0.61	1.17	6874	4657	5427
Average	0.58	0.79	1.34	14,186	7301	45,959
$\alpha = 0.5$						
ftv120	0.00	0.10	0.10	6100	1030	3278
ftv130	0.05	0.05	0.09	6100	926	3154
ftv140	0.40	0.53	0.93	5989	2166	8231
ftv150	0.31	0.08	0.39	6886	1877	9124
ftv160	0.08	0.32	0.40	11,180	3537	8192
ftv170	0.00	0.41	0.41	7905	2966	10,800
rbg323	0.33	1.69	2.01	21,247	19,988	133,136
rbg358	0.33	1.01	1.30	31,325	16,440	130,826
rbg403	0.13	2.34	2.47	40,234	22,991	205,795
rbg443	0.14	1.31	1.45	40,519	31,854	198,349
dc176	1.21	0.36	1.54	9138	1502	3520
dc188	0.82	0.86	1.67	7905	1859	3114
code198	2.32	1.78	4.01	8993	2060	3418
code253	2.46	1.68	4.04	10,589	2805	6437
td316_10	1.42	0.57	1.96	7853	4876	5104
Average	0.67	0.87	1.50	14,798	7792	48,832
$\alpha = 0.8$						
ftv120	0.00	0.15	0.15	8664	1265	2735
ftv130	0.23	0.37	0.60	8664	1545	2973
ftv140	0.40	0.67	1.06	9283	1178	8386
ftv150	0.04	0.12	0.16	8397	1560	8192
ftv160	0.08	0.51	0.59	6827	2381	8208
ftv170	0.00	0.41	0.41	6968	2623	9912
rbg323	0.26	1.44	1.67	24,854	18,887	147,954
rbg358	0.17	0.30	0.47	35,027	18,806	154,563
rbg403	0.11	1.85	1.96	41,682	24,102	229,627
rbg443	0.14	1.31	1.42	41,301	32,059	234,349
dc176	1.61	1.07	2.64	17,521	1791	12,538

Continued

© 2021 The Authors.

International Transactions in Operational Research © 2021 International Federation of Operational Research Societies

752

G. Pantuza and M.C. de Souza / Intl. Trans. in Op. Res. 29 (2022) 729–759

Table 6
Continued

Instance	gap UB	gap LB	gap Δ	Iterations	Time Algo 1	Time PSC (MIP)
dc188	0.99	0.89	1.86	9648	1976	3533
code198	2.32	1.78	4.01	9452	2158	3406
code253	2.77	1.60	4.26	11,552	3763	40,863
td316_10	2.01	0.75	2.71	9749	4973	8892
Average	0.74	0.88	1.59	16,639	7938	58,409

heuristic did not succeed to find an optimal primal solution). Thus, with Algorithm 1 it is possible to bound the optimal values of the larger *ftv* instances within an optimality gap of less than 1%, except for instance *ftv140* $\alpha = 0.8$ whose gap Δ is 1.06%. It is worth mentioning that these results were obtained with considerably less computational effort than needed to obtain optimal solutions running PSC with Gurobi.

The *rbg* are the largest instances. The computational effort needed to obtain optimal solutions for them running PSC with Gurobi was huge, nearly two days of computation (47.7 hours) on average. Algorithm 1, in turn, proved to be an efficient alternative to find near-optimal solutions while spending moderate computational time. In fact, Algorithm 1 presented on average an optimality gap of 1.61% over the group of instances *rbg*, taking 6.3 hours on average. The largest optimality gap was 2.47% for instance *rbg403* $\alpha = 0.5$. The heuristic obtained near-optimal solutions for all the *rbg* instances, gap UB is no greater than 0.51%. The lower bounds provided by the Lagrangian relaxation were tight even for these larger and harder instances. For 10 out of the 12 *rbg* instances gap LB was less than 2% (only for instances *rbg403* $\alpha = 0.2$ and $\alpha = 0.5$ gap LB was 2.34%). The most difficult instances for Algorithm 1 in the group *realworld* were the *code*. The heuristic returned solutions with gap UB greater than 2%, and Algorithm 1 returned optimality gaps around 4%.

Table 7 illustrates the impact of the use of cost-cover and cycle-cover cuts on the optimality gaps obtained by Algorithm 1. We consider the following versions of Algorithm 1: *cc*—both cuts were used (the final version); *co*—only cost-cover cuts were used; *cy*—only cycle-cover cuts were used; *no*—no cuts were used at all. Table 7 shows gap Δ and the number of iterations executed by each version. It can be observed that the use of valid inequalities in a relax-and-cut fashion improved the performance of Algorithm 1. Each cut individually was useful to improve optimality gaps, and, when arc costs allow the use of cost-cover cuts the use of both cuts improved upon each one individually. The convergence was also improved, as the number of iterations was smaller than using no cuts at all.

4.2.2. Group crane and group stilt instances

The instances of group *crane* do not satisfy the triangle inequality, while the instances of group *stilt* do. Thus, the cost-cover cuts were not applied to the instances of group *crane*. Table 8 shows results for the 69 *crane* and *stilt* instances for which formulation EPN solved the linear relaxation problem within the time limit of 10,800 seconds. These groups contain harder instances. Nevertheless, the heuristic obtained good quality solutions, gap UB was no greater than 1% for 49% of the instances (34 out of 69 instances), and its maximum value was 1.88%. The gap LB was

© 2021 The Authors.

International Transactions in Operational Research © 2021 International Federation of Operational Research Societies

Table 7
The impact of the use of cost-cover and cycle-cover cuts on the performance of Algorithm 1

Instance	gap Δ				Iterations			
	Δ_{cc}	Δ_{co}	Δ_{cy}	Δ_{no}	it_{cc}	it_{co}	it_{cy}	it_{no}
$\alpha = 0.2$								
ft53	3.37	9.42	13.34	18.38	4633	6708	4633	9095
ft70	2.57	2.57	2.57	12.31	2708	2708	2944	8033
ftv170	0.41	7.86	7.37	18.31	7905	7918	8092	9705
rbg323	–	–	2.16	3.33	–	–	25,512	27,638
rbg358	–	–	0.40	0.51	–	–	30,215	33,738
rbg403	–	–	2.33	2.47	–	–	38,112	42,539
rbg443	–	–	1.71	1.79	–	–	37,034	41,993
$\alpha = 0.5$								
ft53	4.38	12.09	12.01	16.56	7657	8705	7657	9174
ft70	2.89	10.65	10.65	17.45	2708	2911	2765	4696
ftv170	0.41	0.41	1.47	17.96	7905	7918	8092	8832
rbg323	–	–	1.92	3.16	–	–	21,247	26,755
rbg358	–	–	1.30	1.71	–	–	31,325	32,891
rbg403	–	–	2.38	2.47	–	–	40,234	44,874
rbg443	–	–	1.42	1.42	–	–	40,519	43,522
$\alpha = 0.8$								
ft53	2.60	7.62	7.32	18.67	1083	1919	1083	2119
ft70	2.29	12.61	11.08	17.47	5466	7797	5466	9341
ftv170	0.41	0.41	0.41	16.35	6968	7019	7007	8839
rbg323	–	–	1.67	3.01	–	–	24,854	27,131
rbg358	–	–	0.40	1.11	–	–	35,027	38,689
rbg403	–	–	1.91	5.45	–	–	41,682	45,791
rbg443	–	–	1.42	1.50	–	–	41,301	43,824

2.12% on average. The instances of group *stilt* are harder to obtain good lower bounds. Whereas gap *LB* was no greater than 2% for 30 of the 39 crane instances, it was no greater than 2% for only 6 of the 30 *stilt* instances. The bounds obtained by the Lagrangian relaxation improved upon the linear relaxation bounds for 13 of 39 instances. The average optimality gap was 3.05%. It can be observed that gap Δ presented higher values for the group *stilt*.

Table 9 shows results for the remaining 60 crane and *stilt* instances for which formulation EPN was not able to solve the linear relaxation problem within the time limit of 10,800 seconds. For these instances, we ran formulation PSC with a time limit of 10,800 seconds. Thus, we were not able to obtain optimal solutions for most of them. The last column presents the optimality gap returned by Gurobi after 10,800 seconds of computation running formulation PSC. In Table 9, gap *UB* is computed with respect to the upper bound obtained by PSC when the optimal value is not available. The negative values of gap *UB* for instances *stilt316_17* and *stilt316_19*, $\alpha = 0.5$, and *stilt316_13*, *stilt316_18*, and *stilt316_19*, $\alpha = 0.8$, mean that Algorithm 1 obtained better upper bounds than the ones obtained by PSC after 10,800 seconds of computation. The value of gap *UB* is not computed for instance *stilt316_10* $\alpha = 0.5$ since PSC did not obtain a feasible solution after 10,800 seconds of computation. The value of gap

754

G. Pantuza and M.C. de Souza / Intl. Trans. in Op. Res. 29 (2022) 729–759

Table 8

Results obtained by Algorithm 1 for the 69 crane and stilt instances for which EPN linear relaxation values are available

Instance	gap UB	gap LB	gap LP	gap Δ	it	Time Algo 1	Time PSC (MIP)
$\alpha = 0.2$							
crane66_0	0.30	2.87	−0.84	3.15	3744	104	6
crane66_1	0.00	0.00	−2.17	0.00	3924	109	4
crane66_2	0.00	0.00	−1.44	0.00	3920	112	5
crane100_0	1.34	1.53	0.39	2.83	12,321	333	21
crane100_1	0.39	0.92	0.59	1.30	13,186	347	36
crane100_2	0.95	1.97	0.39	2.90	11,618	314	367
crane100_3	0.40	0.80	0.38	1.19	11,988	324	47
crane100_4	0.99	0.71	0.60	1.68	12,502	329	57
crane100_5	1.05	0.85	0.63	1.88	12,654	333	19
crane100_6	0.00	0.00	0.00	0.00	12,210	330	17
crane100_7	0.00	0.00	0.00	0.00	11,894	313	9
crane100_8	1.29	0.93	0.55	2.19	12,358	334	28
crane100_9	0.97	1.21	0.78	2.16	11,803	319	45
stilt100_0	0.44	3.86	0.16	4.29	18,537	501	165
stilt100_1	1.03	4.95	−2.46	5.92	20,976	552	21,562
stilt100_2	1.05	4.14	−0.19	5.13	21,926	577	1121
stilt100_3	1.02	3.45	−5.70	4.43	21,014	553	23,291
stilt100_4	0.35	3.03	0.72	3.37	20,558	541	197
stilt100_5	0.65	2.92	1.61	3.55	19,418	511	21
stilt100_6	1.11	8.56	0.78	9.56	21,470	565	4530
stilt100_7	0.31	5.83	0.61	6.12	19,314	522	676
stilt100_8	0.48	6.58	1.44	7.02	21,964	578	1959
stilt100_9	0.95	2.72	0.45	3.64	20,976	552	299
Average	0.66	2.51	−0.12	3.14	14,795	394	2369
$\alpha = 0.5$							
crane66_0	0.00	2.45	−0.05	2.45	4824	134	27
crane66_1	1.52	0.52	−0.12	2.01	5460	156	14
crane66_2	0.09	0.74	0.00	0.83	5145	147	65
crane100_0	1.44	2.46	1.81	3.84	15,542	409	401
crane100_1	0.59	2.01	1.51	2.58	15,429	417	108
crane100_2	1.30	1.41	0.92	2.68	17,252	454	325
crane100_3	1.46	1.36	1.10	2.78	16,796	442	51
crane100_4	1.29	1.19	0.37	2.44	16,465	445	3706
crane100_5	0.00	0.00	−0.08	0.00	15,884	418	17
crane100_6	0.46	0.81	0.42	1.26	15,651	423	21
crane100_7	0.00	0.00	−0.05	0.00	16,758	441	7
crane100_8	1.43	0.42	0.18	1.82	16,391	443	54
crane100_9	1.66	2.28	0.38	3.87	16,948	446	457
stilt100_0	1.06	1.27	0.24	2.31	21,497	581	420
stilt100_1	0.98	0.66	0.35	1.62	21,867	591	974
stilt100_2	1.50	1.19	0.46	2.65	22,792	616	268
stilt100_3	1.31	2.72	1.09	3.98	21,571	583	280
stilt100_4	0.60	3.40	1.13	3.98	20,202	546	746
stilt100_5	1.81	4.29	0.94	5.99	21,386	578	1681

Continued

© 2021 The Authors.

International Transactions in Operational Research © 2021 International Federation of Operational Research Societies

Table 8
Continued

Instance	gap UB	gap LB	gap LP	gap Δ	it	Time Algo 1	Time PSC (MIP)
stilt100_6	1.18	3.09	1.05	4.22	24,700	650	1380
stilt100_7	0.52	1.93	0.81	2.44	23,104	608	810
stilt100_8	0.89	2.22	1.25	3.08	23,199	627	644
stilt100_9	1.23	2.40	0.11	3.58	22,755	615	797
Average	0.97	1.69	0.60	2.63	17,462	468	576
$\alpha = 0.8$							
crane66_0	0.49	1.17	−0.06	1.65	6545	187	8
crane66_1	0.77	3.18	1.20	3.91	6876	191	29
crane66_2	1.41	1.39	0.41	2.76	6545	187	18
crane100_0	1.59	0.89	0.48	2.44	19,722	519	74
crane100_1	0.82	2.56	1.66	3.36	18,981	513	53
crane100_2	1.61	2.72	−0.38	4.27	17,784	468	176
crane100_3	1.69	0.83	0.51	2.48	18,848	496	23
crane100_4	1.64	1.50	0.33	3.09	20,254	533	668
crane100_5	0.49	2.39	1.63	2.87	19,228	506	44
crane100_6	0.75	1.26	0.57	1.99	17,612	476	18
crane100_7	0.89	0.78	0.53	1.66	18,130	490	10
crane100_8	1.49	0.01	−0.10	1.48	19,000	500	15
crane100_9	1.87	1.28	0.68	3.09	18,352	496	61
stilt100_0	1.75	3.33	0.92	4.99	24,013	649	961
stilt100_1	1.83	2.53	0.89	4.28	25,916	682	369
stilt100_2	1.63	1.83	0.86	3.41	26,603	719	623
stilt100_3	1.28	4.36	2.60	5.57	28,934	782	272
stilt100_4	0.94	2.35	1.78	3.26	29,488	776	166
stilt100_5	1.88	3.86	1.75	5.63	26,524	698	168
stilt100_6	1.27	3.57	2.62	4.78	30,818	811	634
stilt100_7	0.62	0.90	0.09	1.51	26,159	707	557
stilt100_8	1.15	3.72	0.96	4.81	28,157	761	604
stilt100_9	1.58	3.18	0.89	4.69	26,942	709	160
Average	1.28	2.16	0.91	3.39	20,932	559	248

LB is computed with respect to the best known upper bound when the optimal value is not available.

It is clearly more difficult to close the optimality gap for the *stilt* instances. The values of gap Δ and gap PSC observed for the *stilt* instances are both greater than those observed for the *crane* instances. Nevertheless, Algorithm 1 was efficient to deal with the *crane* and the *stilt* instances. The value of gap Δ was 2.37% on average, and it was greater than 5% only for 2 instances. On the other hand, gap PSC was 6.64% on average, and it was greater than 5% for 28 instances besides 1 instance for which it could not be computed. Algorithm 1 obtained better optimality gaps than running PSC with Gurobi for 72% of the instances (43 out of 60 instances). Considering the *stilt* instances, the value of gap Δ was 2.95% on average, whereas gap PSC was 11.73% on average. The value of gap LB was 1.23% on average, and considering only the *stilt* instances it was 1.86% on average.

756

G. Pantuza and M.C. de Souza / Intl. Trans. in Op. Res. 29 (2022) 729–759

Table 9

Results obtained by Algorithm 1 for the 60 crane and stilt instances for which EPN linear relaxation values are not available

Instance	gap <i>UB</i>	gap <i>LB</i>	gap Δ	<i>it</i>	Time Algo 1	Time PSC (MIP)	gap PSC
$\alpha = 0.2$							
crane316_10	0.78	0.74	1.51	205,694	5413	1595	0.00
crane316_11	0.67	0.59	1.25	228,912	6024	10,802	1.52
crane316_12	0.73	0.56	1.28	220,476	5802	10,800	1.90
crane316_13	1.12	0.05	1.16	213,256	5612	7096	0.00
crane316_14	0.86	0.60	1.45	229,900	6050	10,800	5.80
crane316_15	0.94	0.77	1.69	220,818	5811	10,800	1.90
crane316_16	1.01	0.42	1.42	220,096	5792	10,800	3.79
crane316_17	0.93	0.29	1.21	221,160	5820	10,802	1.15
crane316_18	0.52	0.78	1.29	215,042	5659	10,800	2.02
crane316_19	0.56	0.68	1.23	223,326	5877	7984	0.00
stilt316_10	1.74	0.83	2.53	276,678	7281	10,800	5.25
stilt316_11	1.16	1.44	2.57	262,010	6895	10,802	5.52
stilt316_12	0.67	2.13	2.78	259,806	6837	10,800	7.03
stilt316_13	1.61	2.11	3.66	269,534	7093	10,804	4.26
stilt316_14	1.13	1.03	2.14	266,418	7011	10,800	4.08
stilt316_15	1.01	0.25	1.25	272,270	7165	10,800	6.05
stilt316_16	1.39	1.61	2.96	263,264	6928	10,807	6.88
stilt316_17	1.59	0.95	2.50	266,722	7019	10,800	5.82
stilt316_18	1.33	0.27	1.58	267,634	7043	10,803	6.58
stilt316_19	0.88	3.99	4.83	269,458	7091	10,800	2.61
Average	1.03	1.00	2.01	243,624	6411	10,015	3.61
$\alpha = 0.5$							
crane316_10	1.04	0.96	1.99	211,698	5571	1511	0.00
crane316_11	0.94	0.23	1.15	230,090	6055	10,802	1.42
crane316_12	1.72	0.55	2.23	218,576	5752	10,800	2.55
crane316_13	1.58	0.84	2.39	231,154	6083	10,342	0.00
crane316_14	1.35	0.18	1.51	203,756	5362	10,802	1.30
crane316_15	1.44	0.08	1.50	208,316	5482	10,800	1.85
crane316_16	1.75	0.81	2.52	214,548	5646	10,800	5.57
crane316_17	1.13	1.17	2.28	228,646	6017	10,802	2.09
crane316_18	0.81	0.31	1.10	202,654	5333	10,800	1.87
crane316_19	1.10	0.70	1.78	212,344	5588	5068	0.00
stilt316_10	—	2.47	2.47	265,620	6990	10,800	—
stilt316_11	1.90	0.69	2.54	273,714	7203	10,801	9.39
stilt316_12	0.00	2.21	2.21	261,060	6870	10,800	10.75
stilt316_13	2.05	0.44	2.44	255,702	6729	10,800	6.95
stilt316_14	1.63	1.74	3.31	270,256	7112	10,800	7.66
stilt316_15	1.45	0.73	2.14	270,864	7128	10,800	6.63
stilt316_16	1.87	0.93	2.74	264,328	6956	10,810	9.76
stilt316_17	−0.01	2.46	2.46	266,418	7011	10,800	12.76
stilt316_18	2.13	0.35	2.43	252,092	6634	10,801	7.88
stilt316_19	−0.20	2.31	2.31	273,638	7201	10,800	20.62
Average	1.25	0.94	2.18	240,774	6336	10,027	5.74

Continued

© 2021 The Authors.

International Transactions in Operational Research © 2021 International Federation of Operational Research Societies

Table 9
Continued

Instance	gap UB	gap LB	gap Δ	it	Time Algo 1	Time PSC (MIP)	gap PSC
$\alpha = 0.8$							
crane316_10	1.15	0.89	2.01	211,242	5559	10,802	0.30
crane316_11	0.99	1.85	2.82	204,136	5372	10,800	1.72
crane316_12	1.80	0.17	1.94	207,252	5454	10,800	1.90
crane316_13	1.42	0.89	2.28	230,014	6053	3023	0.00
crane316_14	1.49	1.69	3.13	214,624	5648	10,802	1.85
crane316_15	1.62	0.41	2.00	206,872	5444	10,800	2.15
crane316_16	1.89	0.86	2.70	220,096	5792	10,800	2.46
crane316_17	1.28	0.41	1.66	230,052	6054	10,801	5.45
crane316_18	0.92	0.31	1.22	230,280	6060	10,800	2.00
crane316_19	1.45	0.49	1.91	205,238	5401	9237	0.00
stilt316_10	2.66	0.61	3.18	264,214	6953	10,801	9.25
stilt316_11	0.00	3.58	3.58	258,742	6809	10,800	14.92
stilt316_12	0.05	2.85	2.90	276,184	7268	10,800	10.86
stilt316_13	−0.03	6.03	6.03	267,026	7027	10,800	15.91
stilt316_14	1.86	0.80	2.61	276,754	7283	10,800	7.25
stilt316_15	2.26	0.15	2.36	264,632	6964	10,800	0.72
stilt316_16	1.98	3.73	5.60	266,152	7004	10,800	12.75
stilt316_17	1.25	2.33	3.54	267,596	7042	10,800	11.98
stilt316_18	−0.02	3.67	3.67	260,870	6865	10,800	16.70
stilt316_19	−25.91	3.21	3.21	274,816	7232	10,800	93.33
Average	−0.09	1.75	2.92	241,840	6364	10,333	10.58

5. Concluding remarks

This study has shown that strong formulations can provide tight linear relaxation bounds for the PCTSP. The formulation EPN proposed in this paper dominated adaptations of strong formulations from the literature for the asymmetric TSP in computational experiments based on benchmark instances. However, such formulations require large computational effort to solve the resulting linear relaxation problem, rendering them impractical to obtain optimal solutions with an MIP solver. Thus, we proposed a Lagrangian relaxation approach based on the formulation EPN. The approach embeds heuristic schemes to obtain hopefully optimal or near-optimal solutions and valid cuts from the literature in a relax-and-cut fashion. The heuristic has shown to be successful, obtaining feasible solutions within 1% of optimality in most cases. The proposed Lagrangian relaxation approach, on the other hand, was very efficient to bound optimal values of these large instances in small optimality gaps in moderate computational time.

Since in many cases most of the computing time spent by branch-and-cut methods is consumed to solve linear programs, we believe that a promising line of research is to use the proposed Lagrangian relaxation scheme into a branch-and-cut algorithm. Barahona and Ladányi (2006) has shown that it can be advantageous to replace Simplex with a fast approximate method, that is, the volume algorithm. In their computational experiments, Barahona and Ladányi (2006) have observed that a branch-and-cut where the linear programming relaxations were treated with the volume algorithm was able to solve to optimality substantially more instances of the Steiner and

the max-cut problems than using Simplex. As the linear programs associated with the best formulations for the PCTSP are extremely difficult to solve, we expect that embedding the Lagrangian relaxation into a branch-and-cut algorithm may also be more advantageous.

Acknowledgments

The authors would like to thank the two anonymous referees for helpful suggestions in improving this paper. The second author was partially supported by Brazilian funding agency CNPq, grant 312549/2017-2.

References

- Bahiense, L., Maculan, N., Sagastizabal, C., 2002. The volume algorithm revisited: relation with bundle methods. *Mathematical Programming* 94, 41–69.
- Balas, E., 1989. The prize collecting traveling salesman problem. *Networks* 19, 621–636.
- Balas, E., 1995. The prize collecting traveling salesman problem: II. Polyhedral results. *Networks* 25, 199–216.
- Balas, E., Martin, G., 1985. ROLL-A-ROUND: software package for scheduling the rounds of a rolling mill. Technical Report, Balas and Martin Associates, Pittsburgh.
- Barahona, F., Anbil, R., 2000. The volume algorithm: producing primal solutions with a subgradient method. *Mathematical Programming* 87, 385–399.
- Barahona, F., Ladányi, L., 2006. Branch and cut based on the volume algorithm: Steiner trees in graphs and max-cut. *RAIRO-Operations Research* 40, 53–73.
- Bérubé, J.F., Gendreau, M., Potvin, J.Y., 2009. A branch-and-cut algorithm for the undirected prize collecting traveling salesman problem. *Networks* 54, 56–67.
- Bienstock, D., Goemans, M.X., Simchi-Levi, D., Williamson, D., 1993. A note on the prize collecting traveling salesman problem. *Mathematical Programming* 59, 413–420.
- Chan, T.H.H., Jiang, H. & Jiang, S.H.C., 2020. A unified PTAS for prize collecting TSP and Steiner tree problem in doubling metrics. *ACM Transactions on Algorithms* 16 (March 2020).
- Chaves, A., Lorena, L., 2008. Hybrid metaheuristic for the prize collecting travelling salesman problem. In van Hemert, J., Cotta, C. (eds) *EvoCOP 2008, Lecture Notes in Computer Science*, Vol. 4972. Springer, Berlin, pp. 123–134.
- Cirasella, J., Johnson, D., McGeoch, L., Zhang, W., 2001. The asymmetric traveling salesman problem: algorithms, instance generators, and tests. In Buchsbaum, A., Snoeyink, J. (eds) *Algorithm Engineering and Experimentation. ALENEX 2001, Lecture Notes in Computer Science*, Vol. 2153. Springer, Berlin, pp. 32–59.
- Claus, A., 1984. A new formulation for the travelling salesman problem. *SIAM Journal on Algebraic and Discrete Methods* 5, 21–25.
- Climaco, G., Simonetti, L., Rosseti, I., 2021. A branch-and-cut and MIP-based heuristics for the prize-collecting travelling salesman problem. *RAIRO-Operations Research* 55, S719–S726.
- Dell’Amico, M., Maffioli, F., Sciomachen, A., 1998. A Lagrangian heuristic for the prize collecting travelling salesman problem. *Annals of Operations Research* 81, 289–305.
- Dell’Amico, M., Maffioli, F., Varbrand, P., 1995. On prize-collecting tours and the asymmetric travelling salesman problem. *International Transactions in Operational Research* 2, 297–308.
- Dinitz, Y., 2006. Dinitz’ algorithm: the original version and Even’s version. In Goldreich, O., Rosenberg, A.L., Selman, A.L. (eds) *Theoretical Computer Science: Essays in Memory of Shimon Even*. Springer, Berlin, pp. 218–240.
- Fischetti, M., Gonzalez, J.J.S., Toth, P., 1998. Solving the orienteering problem through branch-and-cut. *INFORMS Journal on Computing* 10, 133–148.
- Fischetti, M., Toth, P., 1988. An additive approach for the optimal solution of the prize-collecting travelling salesman problem. In Golden, B., Assad, A. (eds) *Vehicle Routing: Methods and Studies*. Elsevier, Amsterdam, pp. 319–343.

© 2021 The Authors.

International Transactions in Operational Research © 2021 International Federation of Operational Research Societies

- Fischetti, M., Toth, P., 1997. A polyhedral approach to the asymmetric traveling salesman problem. *Management Science* 43, 1520–1536.
- Frangioni, A., Gendron, B., Gorgone, E., 2017. On the computational efficiency of subgradient methods: a case study with Lagrangian bounds. *Mathematical Programming Computation* 9, 573–604.
- Gimadi, E., Tsidulko, O., 2020. Asymptotically optimal algorithms for the prize-collecting traveling salesman problem on random inputs. In Matsatsinis, N., Marinakis, Y., Pardalos, P. (eds) *LION 2019, Lecture Notes in Computer Science*, Vol. 11968. Springer, Berlin, pp. 201–207.
- Godinho, M., Gouveia, L., Magnanti, T., 2008. Combined route capacity and route length models for unit demand vehicle routing problems. *Discrete Optimization* 5, 350–372.
- Godinho, M.T., Gouveia, L., Pesneau, P., 2010. Natural and extended formulations for the time-dependent traveling salesman problem. CIO Working Paper 8-2010, Universidade de Lisboa, Lisbon.
- Godinho, M.T., Gouveia, L., Pesneau, P., 2011. On a time-dependent formulation and an updated classification of ATSP formulations. In Mahjoub, R. (ed.) *Progress in Combinatorial Optimization*. Wiley, Hoboken, NJ, pp. 223–254.
- Godinho, M.T., Gouveia, L., Pesneau, P., 2014. Natural and extended formulations for the time-dependent traveling salesman problem. *Discrete Applied Mathematics* 164, 138–153.
- Guignard, M., 2003. Lagrangian relaxation. *TOP* 11, 151–228.
- Held, M., Karp, R., 1970. The traveling salesman problem and minimum spanning trees. *Operations Research* 18, 1138–1162.
- Held, M., Karp, R., 1971. The traveling salesman problem and minimum spanning trees: part II. *Mathematical Programming* 1, 6–25.
- Kumar, R., Li, H., 2000. On asymmetric TSP: transformation to symmetric TSP and performance bound. Available at <https://www.researchgate.net/publication/2613243>.
- Lopez, L., Carter, M.W., Gendreau, M., 1998. The hot strip mill production scheduling problem: a tabu search approach. *European Journal of Operational Research* 106, 317–335.
- Lucena, A., 2005. Non delayed relax-and-cut algorithms. *Annals of Operations Research* 140, 375–410.
- Oncan, T., Altinel, I.K., Laporte, G., 2009. A comparative analysis of several asymmetric traveling salesman problem formulations. *Computers and Operations Research* 36, 637–654.
- Pedro, O., Saldanha, R., Camargo, R., 2013. A tabu search approach for the prize collecting traveling salesman problem. *Electronic Notes in Discrete Mathematics* 41, 261–268.
- Roberti, R., Toth, P., 2012. Models and algorithms for the asymmetric traveling salesman problem: an experimental comparison. *European Journal on Transportation and Logistics* 1, 113–133.
- Sarin, S., Sherali, H., Bhootra, A., 2005. New tighter polynomial length formulations for the asymmetric traveling salesman problem with and without precedence constraint. *Operations Research Letters* 33, 62–70.
- Sherali, H.D., Adams, W., 1990. A hierarchy of relaxations between the continuous and convex hull representations for zero-one programming problems. *SIAM Journal of Discrete Mathematics* 3, 411–430.
- Sherali, H.D., Adams, W., 1994. A hierarchy of relaxations and convex hull characteristics for mixed-integer zero-one programming problems. *Discrete Applied Mathematics* 52, 83–106.
- Sherali, H.D., Sarin, S.C., Tsai, P.F., 2006. A class of lifted path and flow-based formulations for the asymmetric traveling salesman problem with and without precedence constraints. *Discrete Optimization* 3, 20–32.
- Shibasaki, R.S., Baiou, M., Barahona, F., Mahey, P., de Souza, M.C., 2021. Lagrangian bounds for large-scale multi-commodity network design: a comparison between volume and bundle methods. *International Transactions in Operational Research* 28, 296–326.
- Tang, L., Wang, X., 2008. An iterated local search heuristic for the capacitated prize-collecting travelling salesman problem. *Journal of the Operational Research Society* 59, 590–599.
- Toth, P., 1980. Dynamic programming algorithms for the zero-one knapsack problem. *Computing* 25, 29–45.
- Zhang, Y., Tang, L., 2005. Color coated coil scheduling in cold rolling plant using artificial neural network. 2005 ICSC Congress on Computational Intelligence Methods and Applications, Istanbul, Turkey.